Non-Recursive InOrder Traversal

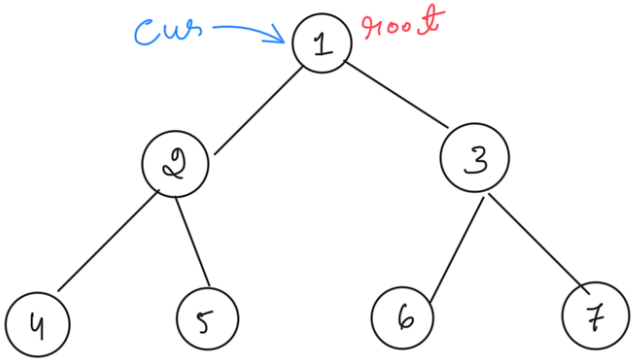
DAY 169
26/Dec/25

The Non-Recursive version of InOrder traversal is similar to PreOrder. The only change is, instead of processing the node Before going to left Subtree, process it after popping (which is indicated after completion of left subtree processing)

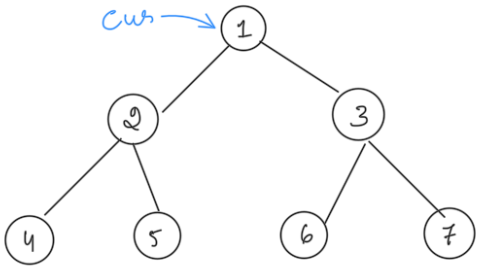
Idea

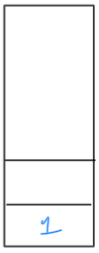
1. Go left as much as possible
2. Use Stack to Remember nodes
3. When null → process node → go right

```
public static void inorder(TreeNode root) {

    Stack<TreeNode> st = new Stack<>();
    TreeNode cur = root;

    while (cur != null || !st.isEmpty()) {

        while (cur != null) {
            st.push(cur);
            cur = cur.left;
        }

        cur = st.pop();
        System.out.print(cur.data + " ");
        cur = cur.right;

    }

}
```

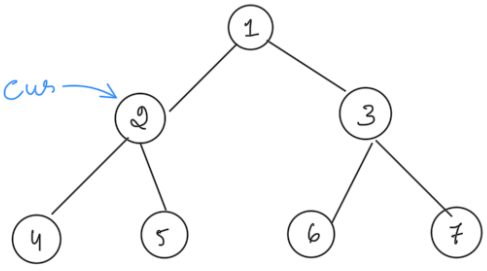


cur != Null
Stack is Empty

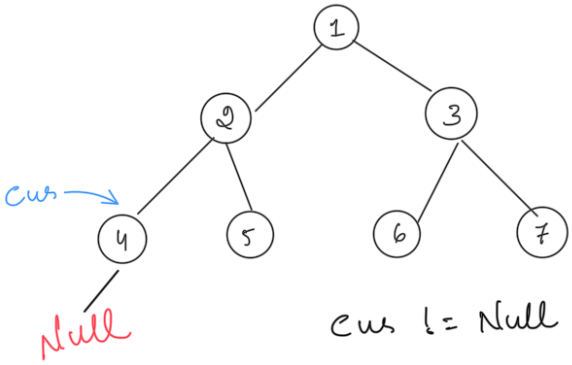


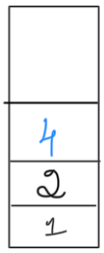


cur != Null



cur != Null



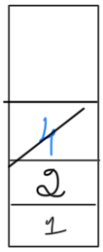


cur != Null

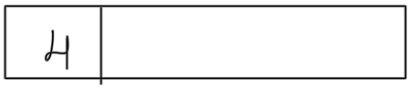
cur.left = Null

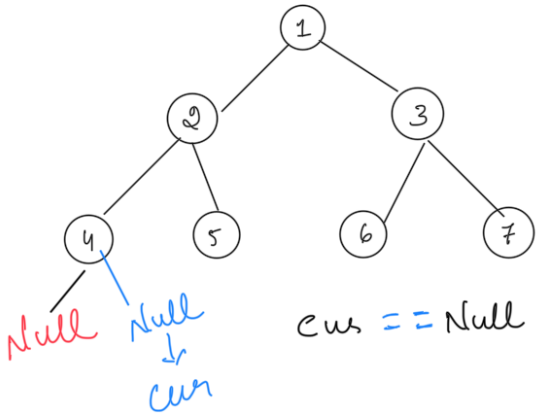
Now pop the stack

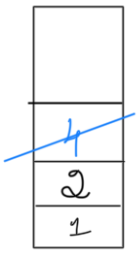
cur = st.pop()



Inorder

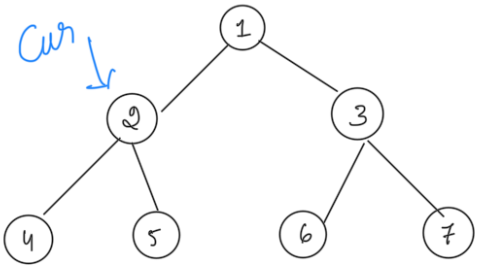




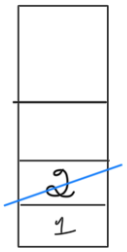


cur == Null

cur = cur.Right = Null



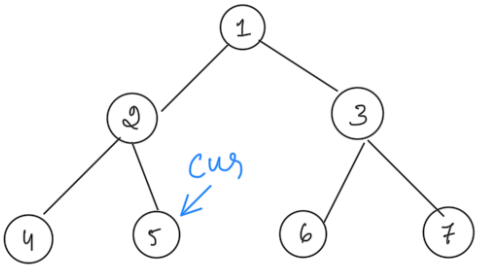
cur != Null



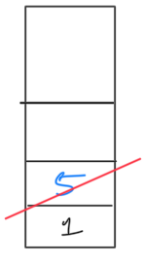
cur = st.pop()

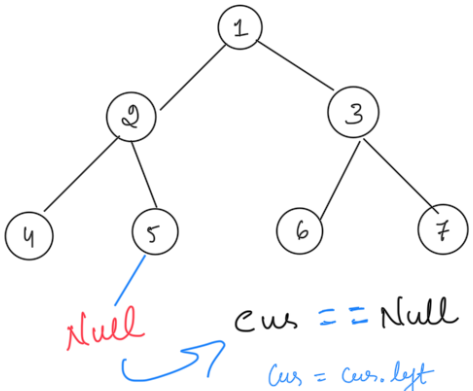
Inorder

cur = cur.right



cur != Null

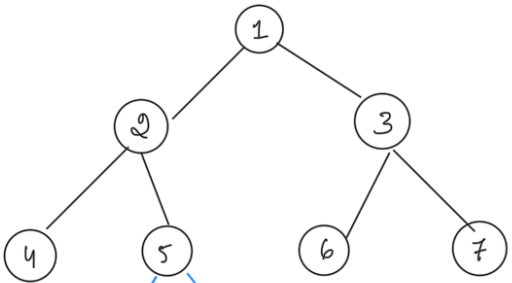




cur == Null

cur = cur.left

cur = Null

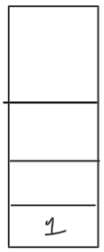


cur == Null

cur = cur.left

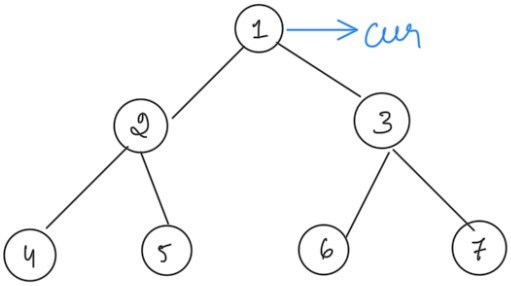
cur = Null

Inorder



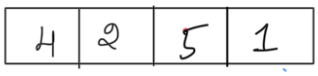
cur = st.pop()

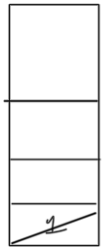
cur = 1



cur != Null

Inorder

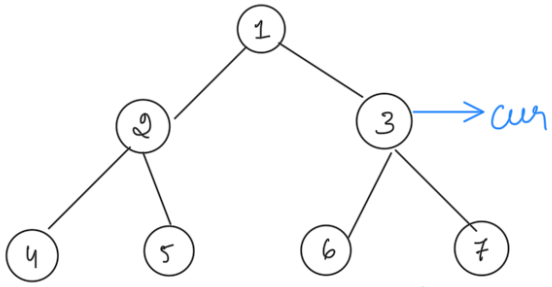




cur = st.pop()

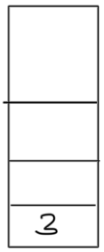
cur = 1

cur = cur.right



cur != Null

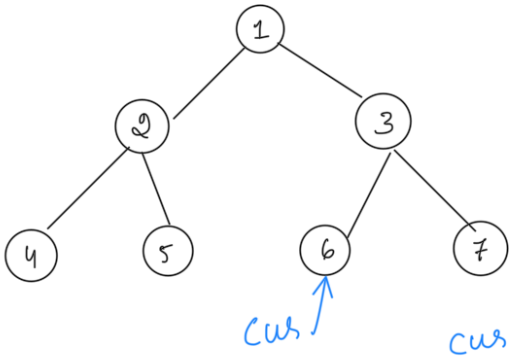
Inorder: 4 2 5 1



cur = st.pop()

cur = 1

cur = cur.right



cur != Null

Inorder: 4 2 5 1 6

cur = st.pop()

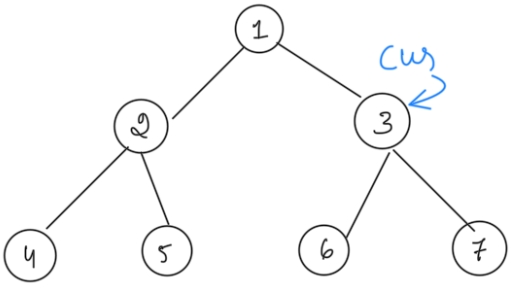
cur = 3.left

cur = cur.right

cur = 6

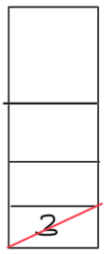
6.left = Null

6.Right = Null



cur != Null

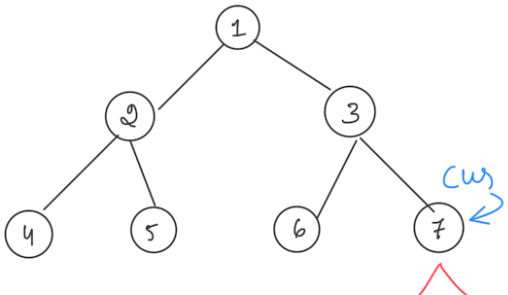
Inorder: 4 2 5 1 6 3



cur = st.pop()

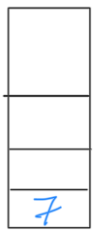
cur = 3.right

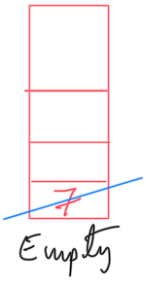
cur = 7



cur != Null

Inorder: 4 2 5 1 6 3 7





cur = st.pop()

cur = Null

Bcz cur.left & cur.right = Null

Loop Breaks cur = Null && Stack is Empty

Time Complexity O(n) Space Complexity O(n)